



Fatima Jinnah Women University

Opening Portals of Excellence Through Higher Education

Lab 2: Hands-On Git and Version Control

DevOps

Name : Eman Khalid

Reg no : 23-BCS-026

Course Code: CS-649

Section: A

GitHub username: EmanKhalid27

Submitted To: Sir Waqas Saleem

Department: Computer Science

1. Introduction

This lab provides hands-on practice with Git and GitHub version control. It covers repository creation, SSH authentication, Git configuration, local repository management, staging and commits, branching, merging, pull requests, code review, branch strategies, and branch cleanup.

2. Objectives

The objectives of this lab are to:

- Understand basic Git and GitHub workflows.
- Configure Git identity and SSH authentication.
- Create, initialize, and manage repositories.
- Track, stage, commit, and push files.
- Create and manage branches.
- Merge branches using Git and GitHub.
- Practice Pull Request and code review workflows.
- Implement a main → staging → develop → feature/bugfix style workflow.
- Apply branch cleanup best practices.

3. Tasks and Evidence

Git Installation:

```
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git --version  
git version 2.53.0
```

Task 1 — Create Private GitHub Repository

A private GitHub repository named **DevOps_Lab2_Practice** was created for practicing the Git workflow.

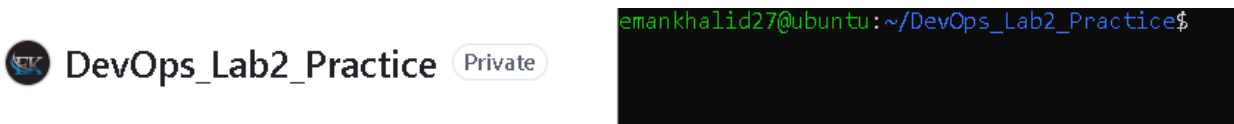


Figure 1: Private GitHub repository

Task 2 — Connect Repository via SSH

An Ed25519 SSH key was generated and the public key was added to GitHub. The repository was then cloned using SSH.

Evidence:

```
emankhalid27@ubuntu:~$ ssh-keygen -t ed25519 -C "emankhalidek.01@gmail.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/emankhalid27/.ssh/id_ed25519):
/home/emankhalid27/.ssh/id_ed25519 already exists.
Overwrite (y/n)?
emankhalid27@ubuntu:~$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIAqRSztCUf3TvU8doO9ig4pvATrqIMW0e6Nejirum2Q4 emankhalidek.01@gmail.com
```

Figure 2: SSH key generation

```
emankhalid27@ubuntu:~$ ssh-keygen -t ed25519 -C "emankhalidek.01@gmail.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/emankhalid27/.ssh/id_ed25519):
/home/emankhalid27/.ssh/id_ed25519 already exists.
Overwrite (y/n)?
emankhalid27@ubuntu:~$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIAqRSztCUf3TvU8doO9ig4pvATrqIMW0e6Nejirum2Q4 emankhalidek.01@gmail.com
emankhalid27@ubuntu:~$
```

Authentication keys



Ubuntu DevOps Lab 2

SHA256:6ZSGtCpDmevGkEDSKRmLMefabhpvJ09/pIv6Azj6/VQ

Added on Oct 3, 2026

Last used within the last week — Read/write

SSH

Delete

Figure 3: SSH public key added to GitHub

```
emankhalid27@ubuntu:~$ git clone git@github.com:EmanKhalid27/DevOps_Lab2_Practice.git
Cloning into 'DevOps_Lab2_Practice'...
warning: You appear to have cloned an empty repository.
emankhalid27@ubuntu:~$
```

Figure 4: Repository cloned using SSH

Task 3 — Configure Git Username and Email

Git username and email were configured globally and verified using Git configuration commands.

Evidence:

```
emankhalid27@ubuntu:~$ git config --global user.name "Eman Khalid"
emankhalid27@ubuntu:~$ git config --global user.email "emankhalidek.01@gmail.com"
emankhalid27@ubuntu:~$
```

Figure 5: Git identity configuration

```
emankhalid27@ubuntu:~$ git config --list
user.name=Eman Khalid
user.email=emankhalidek.01@gmail.com
```

Figure 6: Git configuration verification

Task 4 — Explore the .git Folder

The hidden .git directory was explored using `ls -a .git`. It contains repository configuration, history, logs, and other Git metadata.

```
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ ls -a .git
.  ..  HEAD  config  description  hooks  info  objects  refs
```

Figure 7: Contents of the .git directory

Task 5 — Local Repository Management

The existing .git directory was removed and the repository was reinitialized using git init. A README.md file was created, committed, and pushed to GitHub.

Evidence:

```
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ rm -rf .git
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git status
fatal: not a git repository (or any of the parent directories): .git
```

Figure 8: Removal of existing Git repository

```
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: will change to "main" in Git 3.0. To configure the initial branch name
hint: to use in all of your new repositories, which will suppress this warning,
hint: call:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /home/emankhalid27/DevOps_Lab2_Practice/.git/
```

Figure 9: Git repository initialization

```
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ echo "# Lab02 Git Practice" >> README.md
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git add README.md
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git commit -m "Initial commit"
[master (root-commit) b404a2e] Initial commit
1 file changed, 1 insertion(+)
create mode 100644 README.md
```

Figure 10: First commit

```
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git remote add origin git@github.com:Emankhalid27/DevOps_Lab2_Practice.git
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git branch -M main
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 239 bytes | 59.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:Emankhalid27/DevOps_Lab2_Practice.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Figure 11: First push to GitHub

Task 6 — File Status and Staging

A notes.txt file was created and checked using git status. The file was then staged and committed.

```

emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ echo "Adding some test notes" >> notes.txt
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        notes.txt

nothing added to commit but untracked files present (use "git add" to track)

```

Figure 12: Git status before staging

```

emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git add notes.txt
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git commit -m "Add notes.txt"
[main 6671f9b] Add notes.txt
1 file changed, 1 insertion(+)
create mode 100644 notes.txt

```

Figure 13: Commit of notes.txt

Task 7 — Branch Creation Using GitHub GUI

The branch bugfix/user-auth-error was created through the GitHub web interface and synchronized with the local repository.

Evidence:

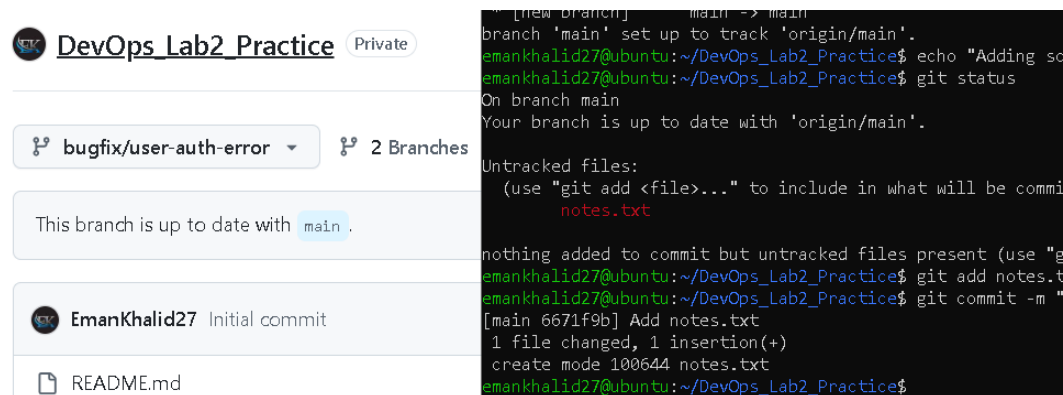


Figure 14: Bugfix branch created using GitHub GUI

```

emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git pull origin bugfix/user-auth-error
From github.com:EmanKhalid27/DevOps_Lab2_Practice
* branch                bugfix/user-auth-error -> FETCH_HEAD
* [new branch]          bugfix/user-auth-error -> origin/bugfix/user-auth-error
Already up to date.

```

Figure 15: Local bugfix branch

Task 8 — Branch Creation and Push Using Git Bash

The branch feature/db-connection was created locally and pushed to the remote GitHub repository.

```

emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git checkout -b feature/db-connection
Switched to a new branch 'feature/db-connection'
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git push origin feature/db-connection
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 3 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 305 bytes | 152.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/db-connection' on GitHub by visiting:
remote:   https://github.com/EmanKhalid27/DevOps_Lab2_Practice/pull/new/feature/db-connection
remote:
To github.com:EmanKhalid27/DevOps_Lab2_Practice.git
 * [new branch]      feature/db-connection -> feature/db-connection

```

Figure 16: Feature database branch creation and push

Task 9 — Branching and Merging

The feature-1 branch was created and a new function was added to main.py. The changes were committed, merged into main, and the branches were pushed to GitHub.

Evidence:

```

emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git checkout -b feature-1
Switched to a new branch 'feature-1'

```

Figure 17: Feature branch creation

```

emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ echo "def new_feature():" >> main.py
hello'" >> main.py
git add main.py
git commit -m "Aemankhalid27@ubuntu:~/DevOps_Lab2_Practice$ echo "    return 'hello'" >> main.py
Add new function to main.py"emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git add main.py
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git commit -m "Add new function to main.py"
[feature/db-connection 812852d] Add new function to main.py
 1 file changed, 2 insertions(+)
 create mode 100644 main.py

```

Figure 18: Feature branch commit

```

emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git checkout main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git merge feature-1
Already up to date.

```

Figure 19: Branch merge

```

emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git push origin main
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:Emankhalid27/DevOps_Lab2_Practice.git
   b404a2e..6671f9b  main -> main
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git push origin feature-1
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature-1' on GitHub by visiting:
remote:   https://github.com/Emankhalid27/DevOps_Lab2_Practice/pull/new/feature-1
remote:
To github.com:Emankhalid27/DevOps_Lab2_Practice.git
 * [new branch]      feature-1 -> feature-1

```

Figure 20: Branches pushed to GitHub

Task 10 — Pull Request and Branch Review

A Pull Request was created from feature/db-connection to main. The Pull Request was reviewed and merged through GitHub, after which the feature branch was deleted.

The image shows a GitHub Pull Request interface on the left and a terminal window on the right. The Pull Request is titled 'feature/db-connection' and is ready to be merged. It shows two commits: 'Add new function to main.py' and 'Add database connection config'. The terminal window shows the commands used to create the pull request, including 'git checkout feature/db-connection', 'git commit -m "Add database connection config"', and 'git push origin feature/db-connection'.

GitHub Pull Request Interface:

- base: main, compare: feature/db-connection, Able to merge. These branches can be automatically merged.
- Discuss and review the changes in this comparison with others. [Learn about pull requests](#). [Create pull request](#)
- 2 commits, 2 files changed, 1 contributor
- Commits on Oct 4, 2026
- Add new function to main.py (Emankhalid27 committed 7 minutes ago)
- Add database connection config (Emankhalid27 committed 2 minutes ago)
- Showing 2 changed files with 3 additions and 0 deletions
- db_config.txt: 1 + db_host=localhost
- main.py: 1 + def new_feature():, 2 + return 'hello'

Terminal Window:

```

Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Create a pull request for 'feature-1' on GitHub by visiting:
remote:   https://github.com/Emankhalid27/DevOps_Lab2_Practice/pull/new/feature-1
remote:
To github.com:Emankhalid27/DevOps_Lab2_Practice.git
 * [new branch]      feature-1 -> feature-1
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git checkout feature/db-connection
Switched to branch 'feature/db-connection'
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ echo "db_host=localhost" >> db_config.txt
db_config.txt
git commit -m "Add database connection config" emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git add db_config.txt
[feature/db-connection 6c6cf48] Add database connection config
1 file changed, 1 insertion(+)
create mode 100644 db_config.txt
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git push origin feature/db-connection
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 3 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (6/6), 606 bytes | 121.00 KiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To github.com:Emankhalid27/DevOps_Lab2_Practice.git
   6671f9b..6c6cf48  feature/db-connection -> feature/db-connection
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$

```

Figure 21: Pull Request creation

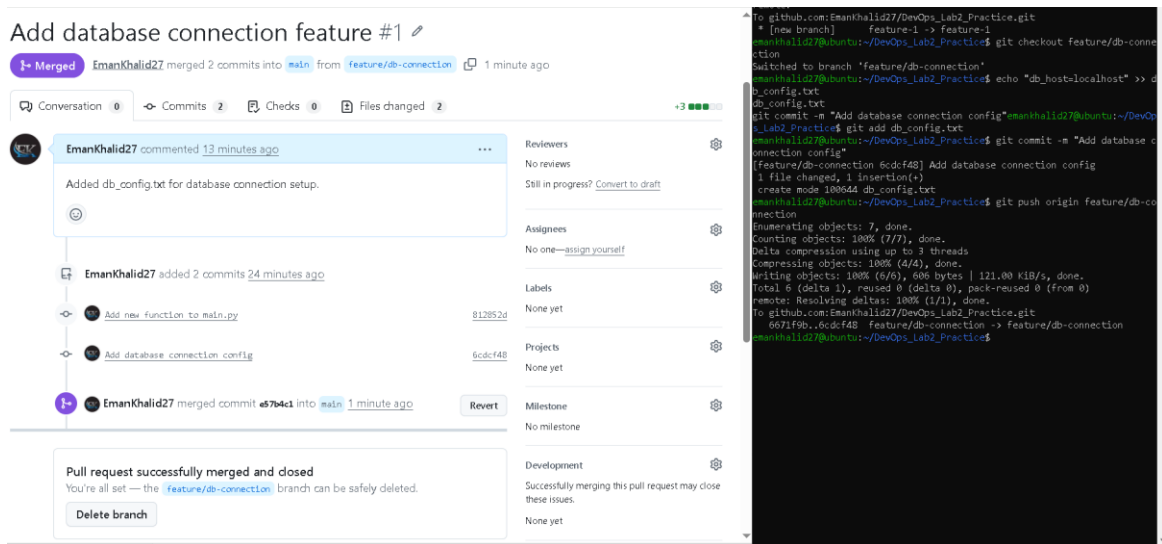


Figure 22: Pull Request review and merge

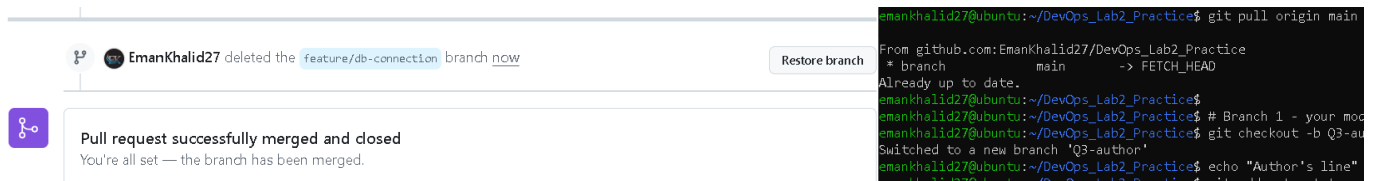


Figure 23: Feature branch deletion

Task 11 — Detailed Branch Strategy

A professional branching strategy was implemented using:

- main
- staging
- develop
- feature/*
- bugfix/*

The workflow followed was:

feature/bugfix → develop → staging → main

The branches feature/lab2-strategy and bugfix/lab2-strategy were used to demonstrate the workflow.

Evidence:

```

emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git branch -a
bugfix/lab2-strategy
develop
feature-1
feature/db-connection
feature/lab2-strategy
* main
staging
remotes/origin/bugfix/lab2-strategy
remotes/origin/bugfix/user-auth-error
remotes/origin/develop
remotes/origin/feature-1
remotes/origin/feature/db-connection
remotes/origin/feature/lab2-strategy
remotes/origin/main
remotes/origin/staging

```

Figure 24: Professional branch strategy

```

emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git log --oneline --graph --decorate --all -20
* 5832d68 (HEAD -> main, origin/staging, origin/main, origin/develop, staging, develop) Resolve merge conflict: keep both bugfix and feature notes
* 4ee9856 (origin/feature/lab2-strategy, feature/lab2-strategy) Add feature branch workflow note
* a6eb46a (origin/bugfix/lab2-strategy, bugfix/lab2-strategy) Add bugfix branch workflow note
* e57b4c1 Merge pull request #1 from Emankhalid27/feature/db-connection
* 6cdc4f8 (origin/feature/db-connection, feature/db-connection) Add database connection config
* 812852d Add new function to main.py
* 6671f9b (origin/feature-1, feature-1) Add notes.txt
* b404a2e (origin/bugfix/user-auth-error) Initial commit

```

Figure 25: Branch merge history

```

emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git checkout main
git push origin mainSwitched to branch 'main'
Your branch is up to date with 'origin/main'.
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git pull origin main
From github.com:Emankhalid27/DevOps_Lab2_Practice
* branch      main      -> FETCH_HEAD
Already up to date.
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git merge staging
Updating e57b4c1..5832d68
Fast-forward
 notes.txt | 3 ++-
 1 file changed, 2 insertions(+), 1 deletion(-)
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git push origin main
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:Emankhalid27/DevOps_Lab2_Practice.git
 e57b4c1..5832d68  main -> main
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git branch -a
bugfix/lab2-strategy
develop
feature-1
feature/db-connection
feature/lab2-strategy
* main
staging
remotes/origin/bugfix/lab2-strategy
remotes/origin/bugfix/user-auth-error
remotes/origin/develop
remotes/origin/feature-1
remotes/origin/feature/db-connection
remotes/origin/feature/lab2-strategy
remotes/origin/main
remotes/origin/staging

```

Figure 26: Final merge into main

The README specifically requires documentation of the branch list, merges into develop and staging, and the final merge into main.

Task 12 — Code Review Workflow

A Pull Request workflow was demonstrated using a feature branch. The process included:

1. Creating a Pull Request with a clear title and description.
2. Assigning a reviewer.
3. Requesting changes and comments.
4. Demonstrating a rejected/closed practice Pull Request.
5. Updating the same branch with corrective commits.
6. Reviewing and approving the Pull Request.
7. Merging the Pull Request.
8. Deleting the merged branch.

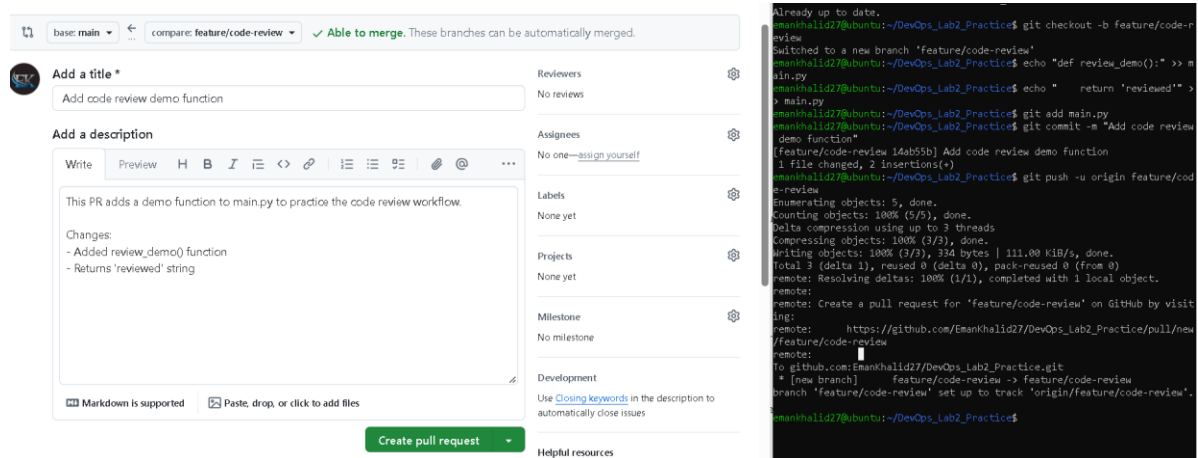


Figure 27: Pull Request details

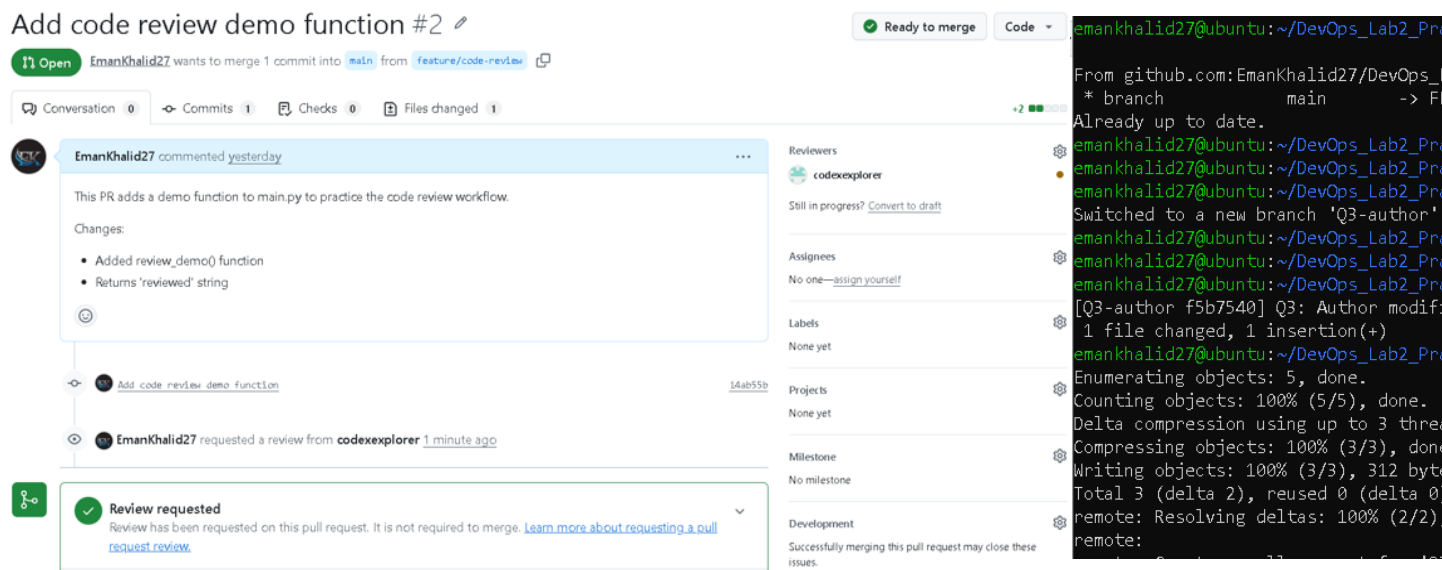


Figure 28: Reviewer assignment

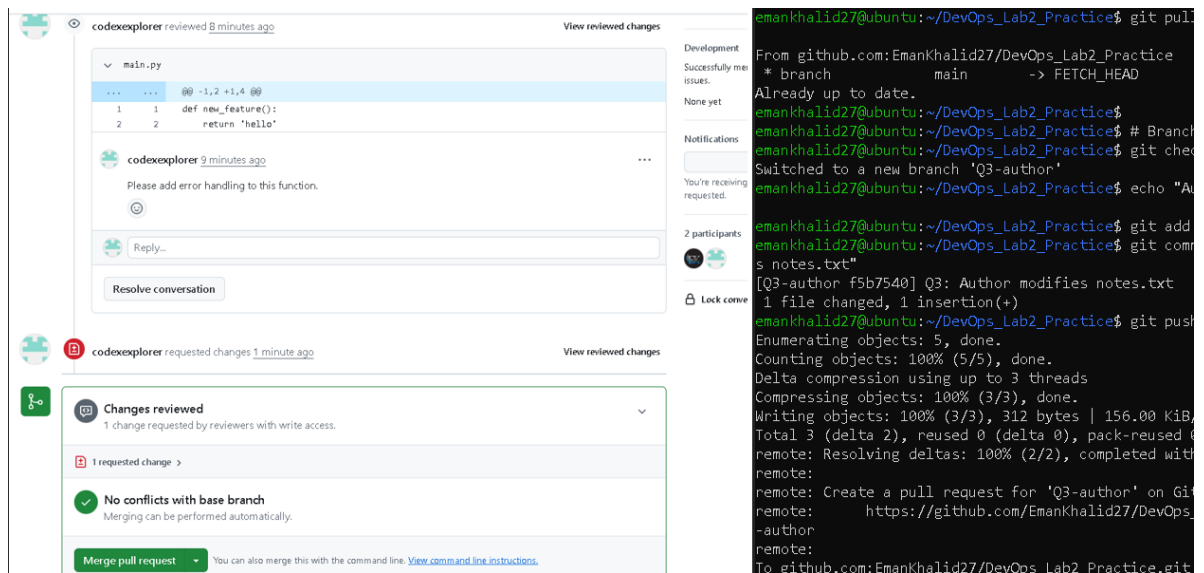


Figure 29: Requested changes

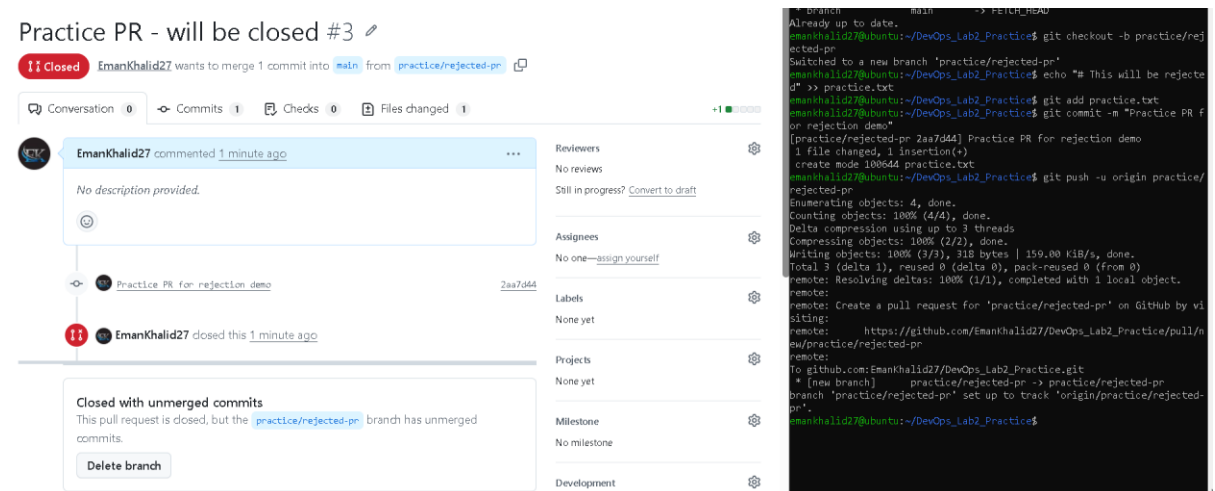


Figure 30: Rejected practice Pull Request

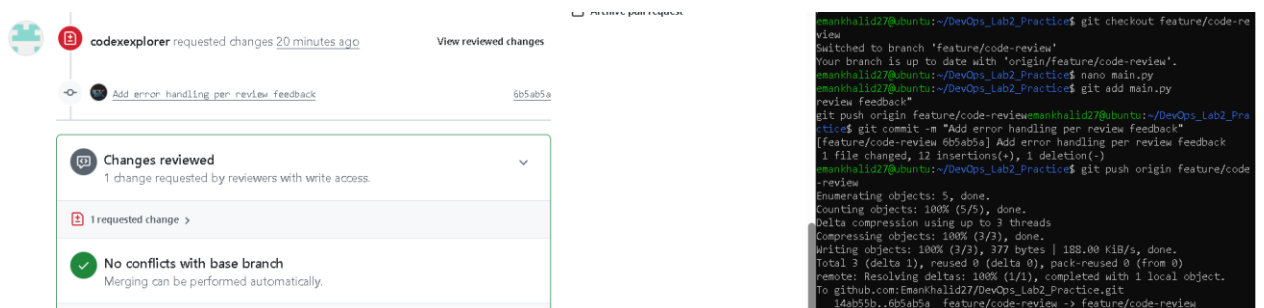
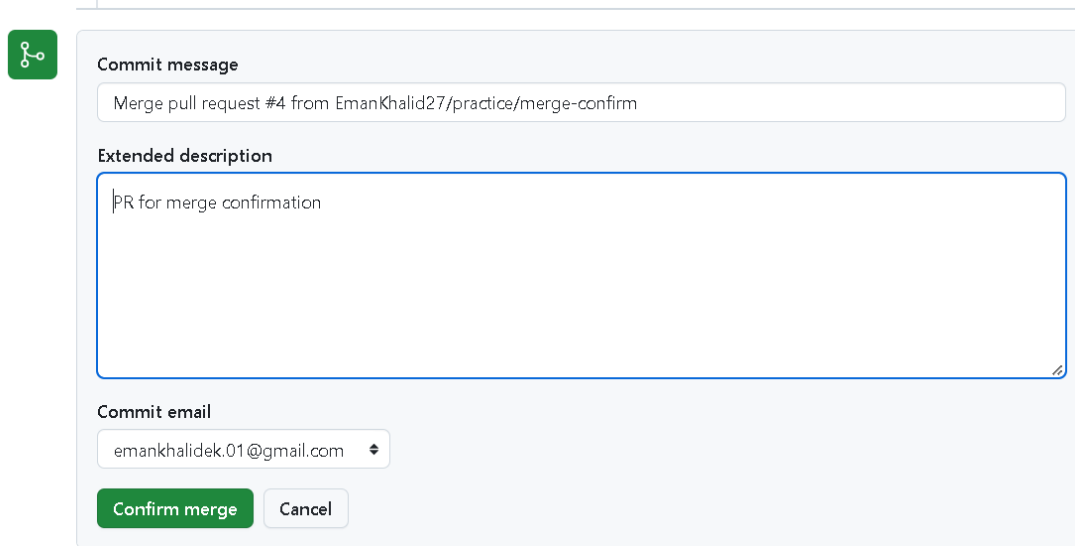


Figure 31: Pull Request updated with new commits



A screenshot of a pull request merge confirmation dialog. It features a green icon with a branching diagram in the top left. The dialog has three main sections: 'Commit message' with a text input containing 'Merge pull request #4 from EmanKhalid27/practice/merge-confirm', 'Extended description' with a large text area containing 'PR for merge confirmation', and 'Commit email' with a dropdown menu showing 'emankhalidek.01@gmail.com'. At the bottom are two buttons: 'Confirm merge' in green and 'Cancel' in white.

Commit message

Merge pull request #4 from EmanKhalid27/practice/merge-confirm

Extended description

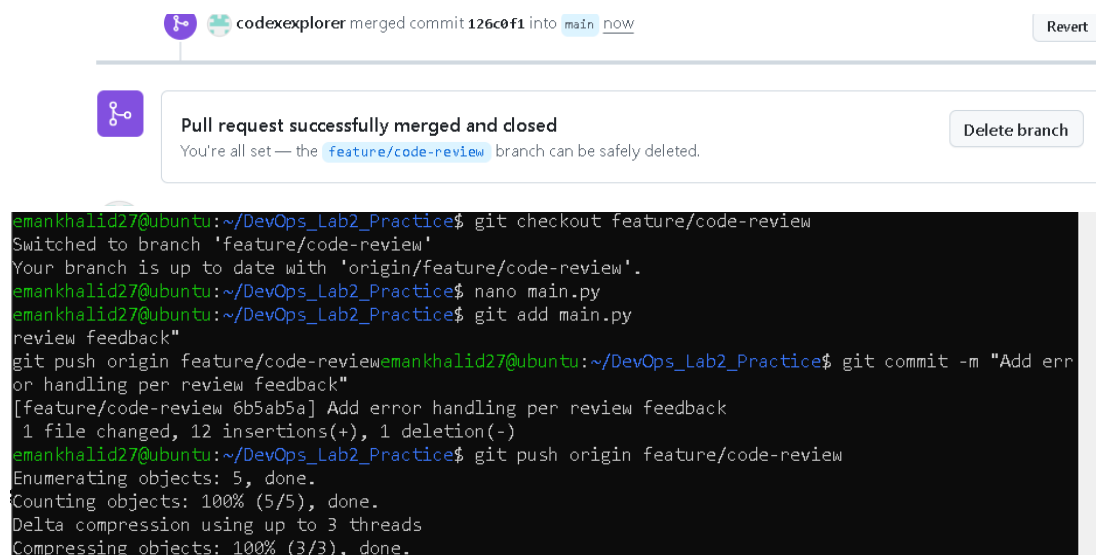
PR for merge confirmation

Commit email

emankhalidek.01@gmail.com

Confirm merge **Cancel**

Figure 32: Pull Request merge confirmation



A screenshot showing a merged pull request notification and a terminal window. The notification at the top shows 'codexexplorer merged commit 126c0f1 into main now' with a 'Revert' button. Below it is a message 'Pull request successfully merged and closed' with a 'Delete branch' button. The bottom part is a terminal window showing the steps to checkout, edit, commit, and push the changes.

codexexplorer merged commit 126c0f1 into main now **Revert**

Pull request successfully merged and closed **Delete branch**

You're all set — the `feature/code-review` branch can be safely deleted.

```
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git checkout feature/code-review
Switched to branch 'feature/code-review'
Your branch is up to date with 'origin/feature/code-review'.
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ nano main.py
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git add main.py
review feedback"
git push origin feature/code-reviewemankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git commit -m "Add error handling per review feedback"
[feature/code-review 6b5ab5a] Add error handling per review feedback
 1 file changed, 12 insertions(+), 1 deletion(-)
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git push origin feature/code-review
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 3 threads
Compressing objects: 100% (3/3), done.
```

Figure 33: Merged Pull Request

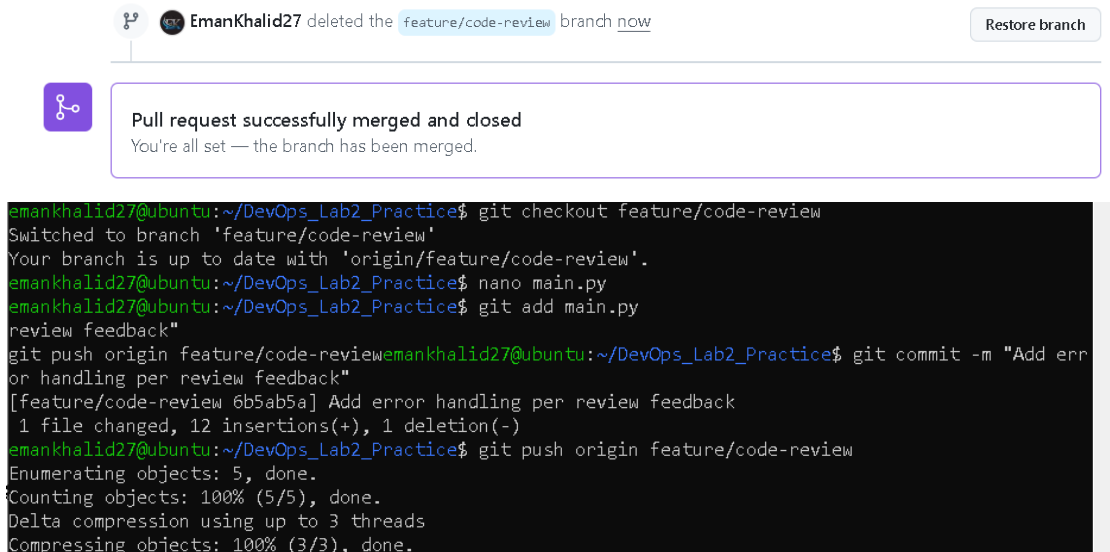


Figure 34: Deleted Pull Request branch

These are the review stages specified in the lab instructions.

Task 13 — Branch Cleanup Best Practices

Remote branches were deleted using both GitHub's web interface and the Git command line.

The terminal deletion command used was:

```
git push origin --delete <branch-name>
```

The local repository was then updated and obsolete local branches were removed.

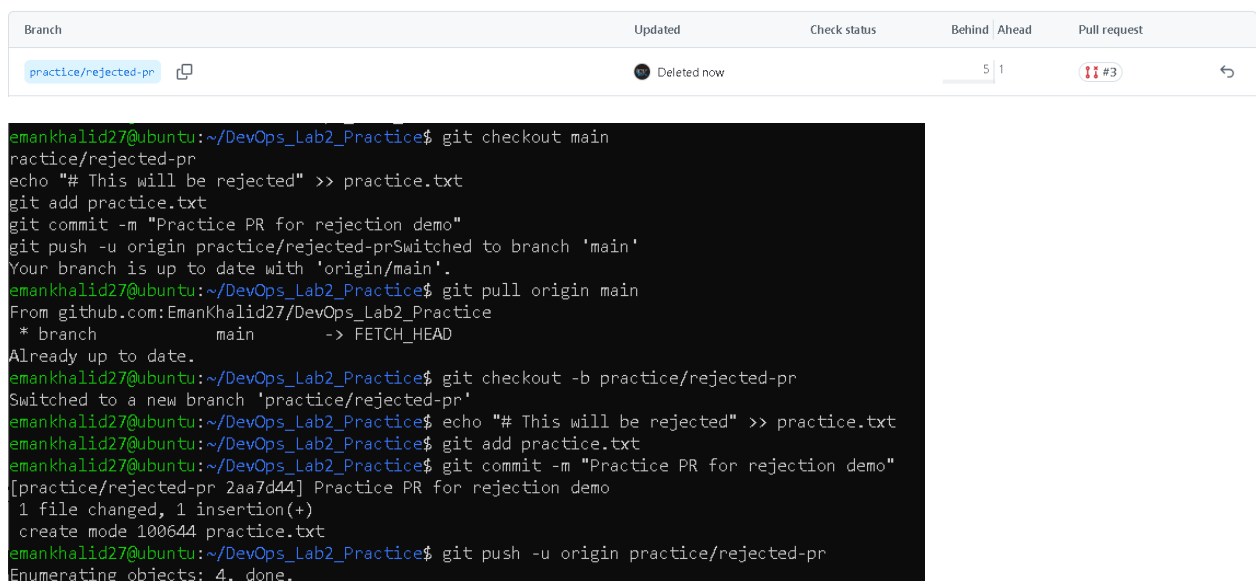


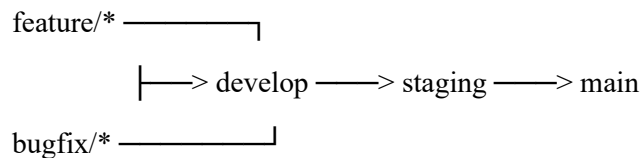
Figure 35: Remote branch deleted through GitHub

```
emankhalid27@ubuntu:~/DevOps_Lab2_Practice$ git push origin --delete feature/lab2-strategy
To github.com:EmanKhalid27/DevOps_Lab2_Practice.git
- [deleted]          feature/lab2-strategy
```

Figure 36: Remote branch deleted using Git

4. Git Branching Workflow

The branching strategy used in this lab can be summarized as:



This workflow separates development, testing, and production branches and provides a structured approach for integrating changes.

5. Conclusion

This lab provided practical experience with Git and GitHub version control. Repository management, SSH authentication, commits, branching, merging, Pull Requests, code review, professional branch strategies, and branch cleanup were successfully practiced. The lab also demonstrated how Git workflows can be organized for collaborative software development.